MASTER'S THESIS DEFENSE REPORT

Explainable AI Clinical Decision Support for Orthopedic Outpatient Care

An encounter-centered web platform and mobile companion application for longitudinal orthopedic care, with a clinician-in-the-loop machine-learning treatment-recommendation pipeline.

CANDIDATE
[Student Name]

SUPERVISOR
[Supervisor Name]

PROGRAMME
[Master's Programme]

INSTITUTION
[University Name]

CLINICAL HOST SITE
Orthopedic Department, Al-Rasoul Hospital (pilot)

DEFENSE DATE
September 2026

SYSTEM NAME
OPD AI Unit — web platform & mobile companion

PILOT WINDOW
March 2026 – September 2026

Repositories referenced in this report: `Orthopedic OPD AI` (backend API, clinical web frontend, ML research pipeline) and `opd-mobile-app` (Expo/React Native patient companion, deployed as an intentionally separate codebase).

Abstract

Orthopedic outpatient departments run high-volume clinics in which a physician must, within a few minutes of consultation time, synthesize demographic data, pre-visit patient-reported outcome measures (PROMs), imaging and laboratory results, prior treatment history, and free-text clinical notes. In the clinic studied here, this information existed but was fragmented across tables related only by a bare patient identifier, with no formal boundary separating what was known *before* a treatment decision from what was recorded *after* it — a structural gap that makes both consistent clinical review and safe predictive modeling difficult.

This thesis presents the design, implementation, and evaluation of the **OPD AI Unit**: a bilingual (Arabic/English) clinical web platform, a companion Expo/React Native mobile application for patients, and an *encounter-centered* longitudinal data model intended to support a future explainable, clinician-in-the-loop treatment-recommendation service. The work follows a Design Science Research Methodology (DSRM), iterating through problem identification, artifact design and development, demonstration in a live pilot clinic, and staged evaluation. Delivered artifacts include: a FastAPI/SQLAlchemy backend of twenty-one routers and twenty-four data models; a React clinical frontend covering pre-visit PROM capture (KOOS-JR, HOOS-JR, QuickDASH, ODI, NDI, SEFAS), physician evaluation, diagnostics, treatment, and surgery workflows; an AI-assisted one-take clinical dictation pipeline combining OpenAI speech transcription with a deterministic, trilingual (English/Arabic/French) keyword-matching safety layer and an LLM structured-extraction fallback; an AI-generated plain-language patient visit-summary feature; and a mobile companion app that authenticates patients by

medical record number and one-time password and re-reads the same backend used by clinical staff, with zero duplicated business logic.

A parallel research track introduces an *Encounter* entity as the unit of analysis for machine learning, a de-identified research export endpoint, and an offline CatBoost-based multi-label training pipeline with explicit leakage-prevention and minimum-sample safety guards. As of this writing, that pipeline has been verified end-to-end at the engineering level — including its deliberate refusal to train below thirty labeled encounters — but has **not yet been trained on a clinically sufficient, clinician-reviewed dataset**; the production recommendation endpoint therefore correctly reports an unavailable state rather than fabricating a suggestion. This thesis treats that outcome as a legitimate, safety-relevant finding in its own right, and contributes a concrete, literature-grounded roadmap — sample-size targets, calibration and subgroup-auditing requirements, and a silent-mode validation protocol — for the clinical deployment phase that follows. The report closes with a discussion of the system's current limitations and a staged plan for future work.

Keywords: clinical decision support systems, explainable artificial intelligence, patient-reported outcome measures, electronic health records, orthopedics, CatBoost, design science research, mobile health, clinical natural language processing.

CHAPTER 1

Introduction

1.1 Background and Clinical Context

Orthopedic outpatient departments (OPDs) sit at the intersection of high patient throughput and information-dense decision-making. A single consultation slot may need to account for a patient's demographic profile, chief complaint, pre-visit questionnaire responses, pain scores, prior diagnoses, imaging and laboratory orders, previous treatments and their outcomes, and any documents uploaded since the last visit. In many clinics, including the host site for this project, this information is not absent — it is simply *distributed*: stored across separate paper forms and disconnected digital records that share little beyond a patient identifier, with no consistent way to reconstruct what was known at the moment a specific clinical decision was made.

The host clinical site — the Orthopedic Department of a partner hospital referred to throughout the deployed application by its own branding as **Al-Rasoul Hospital** — historically relied on a mixture of paper pre-visit forms, informal patient tracking, and a general-purpose records system not tailored to the orthopedic outpatient pathway (pre-visit PROM → physician review → diagnosis → diagnostics → treatment → follow-up). Formal engagement to access and digitize the department's patient records was itself a governance step in this project, evidenced by a written request for database access submitted to the department (*"Request for Access Permission to the Orthopedic Department Patient Database"*) prior to development, establishing the institutional relationship under which the pilot was built.

1.2 Problem Statement

Two related problems motivate this thesis. First, an *operational* problem: the clinic lacked a unified digital workflow spanning patient registration, standardized bilingual PROM capture, physician documentation, diagnostic ordering, treatment recording, and patient-facing follow-up — forcing staff to re-enter or re-derive information at each step and leaving patients with no direct visibility into their own visit history between appointments. Second, a *data-architecture* problem with downstream research consequences: even where digital records existed, they were linked only by a bare patient identifier, with no encounter (visit) boundary, no reliable timestamping of when information became available relative to a treatment decision, and no structured way to distinguish pre-decision information from outcomes recorded later. This second problem is not cosmetic — it is precisely the condition that makes naive machine learning on clinical data dangerous, because a model trained without a decision boundary can silently learn from information that would not have existed at prediction time (a phenomenon this thesis returns to at length in Chapter 7 as *data leakage*).

1.3 Research Motivation

The motivating hypothesis of this work is that solving the operational problem and the data-architecture problem together — rather than sequentially — creates the precondition for a third, higher-value capability: an explainable clinical decision-support assistant that can summarize a patient's current problem, surface comparable historical patterns, and rank commonly selected treatment options for physician review, without displacing clinical judgment. Such an assistant is only trustworthy if it is built on a dataset that is provably free of information the physician would not yet have had, and only useful if its explanations, confidence, and abstentions are visible to the clinician rather than hidden behind a single

opaque score. The system therefore had to be designed, from its data model upward, around an explicit encounter boundary and an audit trail — not retrofitted onto one after the fact.

1.4 Research Questions

1. **RQ1** — Can a reliable, encounter-centered longitudinal dataset be constructed from real orthopedic outpatient records collected through an actively used clinical system, without disrupting clinical workflow?
2. **RQ2** — Can pre-visit patient-reported outcome data, alongside demographic and prior-history features, plausibly improve prediction of the treatment option a clinician selects during an encounter, relative to demographic information alone?
3. **RQ3** — Among a majority-class baseline, regularized logistic regression, random forest, and CatBoost, which model architecture offers the best balance of predictive performance, calibration, and interpretability on this kind of tabular clinical data?
4. **RQ4** — Can model explanations, confidence, and abstention be surfaced to a physician inside an existing clinical workflow in a way that is auditable and does not encourage over-reliance?
5. **RQ5** — What governance, sample-size, and validation conditions must be satisfied before such a model can be safely promoted from an offline research artifact to a live clinical suggestion?

RQ1, RQ4, and RQ5 are substantially answered by the delivered system and are discussed with evidence in Chapters 4 through 8. RQ2 and RQ3 require a clinically sufficient labeled dataset that had not yet accumulated at the time of writing; Chapter 7 reports the engineering-level validation completed to date and specifies, rather than fabricates, the conditions under which RQ2 and RQ3 can be answered.

1.5 Research Objectives

PRIMARY OBJECTIVE

Design, build, and pilot-deploy an explainable, clinician-in-the-loop AI assistant architecture that ranks observed orthopedic treatment choices using only pre-decision clinical data, embedded inside a real physician workflow rather than a disconnected research prototype.

SECONDARY OBJECTIVES

- Design and implement an encounter-centered longitudinal clinical data model spanning pre-visit assessment, physician evaluation, diagnostics, treatment, documents, and follow-up.
- Deliver a full bilingual (Arabic/English) clinical web platform covering the real end-to-end outpatient pathway, not only the research-relevant subset.
- Deliver a mobile companion application giving patients direct, secure access to their own visit history, PROM trends, documents, and clinic messaging.
- Build a reproducible, de-identified research dataset export and an offline model-training pipeline with explicit leakage controls and minimum-sample safety guards.
- Compare baseline models against CatBoost once sufficient data exists, and specify the calibration and subgroup evaluation protocol in advance.
- Instrument an audit trail capturing every model suggestion and every clinician response to it (accept, edit, reject, or “not enough information”).

- Introduce an AI-assisted clinical dictation feature and an AI-generated patient-facing visit summary as near-term, already-functioning applications of the same underlying language-model infrastructure, distinct from the treatment-recommendation model.

1.6 Scope and Boundaries

The system is scoped as *clinical decision support*, not an autonomous diagnostic or prescribing device: at no point does it automatically order a diagnostic test, automatically save a treatment, or hide a low-confidence result. The pilot covers a single hospital department across the full range of body areas the clinic treats (knee, hip, shoulder/elbow/wrist/hand, spine/neck, and foot/ankle), over a development-and-pilot window from March 2026 to September 2026. The treatment-recommendation model is explicitly scoped to predicting *observed clinician treatment choices* from pre-decision information — it is not scoped, at this stage, to proving which treatment is causally optimal, a materially harder claim requiring outcome data this thesis discusses but does not yet claim to have validated at scale.

1.7 Significance and Contributions

1. An encounter-centered orthopedic clinical data model, implemented as a production schema rather than a paper proposal.
2. A complete, bilingual, role-based clinical web platform in active pilot use, covering registration through follow-up.
3. A cross-platform mobile companion application that reuses the same backend and scoring logic as the web platform with no duplicated business logic — itself a validation that the underlying API design generalizes to a second, independently built client.
4. A safety-first AI dictation pipeline combining deterministic multilingual keyword matching with LLM-based structured extraction, designed so that a fixed-vocabulary clinical field is always won by the clinician's own word rather than a model's paraphrase.
5. A reproducible, de-identified longitudinal research-dataset export and an offline, leakage-aware, minimum-sample-gated model training pipeline for treatment-choice prediction.
6. An auditable clinician-feedback loop (accept / edit / reject / insufficient information) wired directly into the physician's existing evaluation screen.
7. A literature-grounded, clinically actionable deployment roadmap for the predictive layer, produced precisely because that layer is not yet trained on sufficient data — turning an honest negative result into a concrete contribution rather than an omission.

1.8 Thesis Structure

Chapter 2 reviews related work in clinical decision support, encounter-based EHR modeling, orthopedic PROMs, explainable tabular machine learning, clinical speech recognition, and mobile health. Chapter 3 details the Design Science Research Methodology followed throughout the project. Chapter 4 covers system analysis, requirements, and architecture. Chapters 5 and 6 describe the web platform and mobile application implementations respectively. Chapter 7 is devoted to the AI clinical decision-support module — its design, its current implementation status, and a set of clinical deployment recommendations. Chapter 8 covers testing and evaluation strategy. Chapter 9 discusses results against the stated objectives. Chapter 10 concludes with a staged plan for future work.

CHAPTER 2

Related Work

This chapter situates the OPD AI Unit within six overlapping bodies of work: clinical decision support systems, encounter-based EHR data modeling, orthopedic patient-reported outcome measures, explainable machine learning on tabular clinical data, clinical speech recognition, and mobile health applications — closing with the specific gap this thesis addresses.

2.1 Clinical Decision Support Systems

Clinical decision support systems (CDSS) have long promised to reduce cognitive load and improve consistency at the point of care, but adoption research consistently identifies workflow disruption, alert fatigue, and opacity as recurring failure modes [1]. Sutton et al. characterize successful CDSS as those embedded directly inside existing clinician workflows rather than offered as a separate destination, and stress that recommendation quality is inseparable from how well the supporting data pipeline reflects real practice [1]. Topol's account of AI in medicine similarly argues that the highest-value near-term role for machine learning is augmenting, not replacing, clinician judgment, particularly in specialties like orthopedics where physical examination and patient context are difficult to fully encode [2]. Rajkomar, Dean, and Kohane note that most clinical ML research is evaluated on retrospective, disconnected datasets rather than inside the live systems clinicians actually use, which limits how much can be inferred about real-world safety or utility [3]. This thesis responds directly to that critique by building the assistant into an actively used pilot system rather than a standalone notebook.

2.2 Encounter-Centered Electronic Health Record Design

The notion of an *encounter* as the atomic unit of a clinical episode is well established in health-information standards; HL7's FHIR specification defines an Encounter resource precisely to bound "an interaction between a patient and healthcare provider(s) for the purpose of providing healthcare service(s)", distinct from the patient-level record it is nested under. The OPD AI Unit's `Encounter` model (Chapter 4) adopts the same conceptual boundary for a system-specific, lightweight reason: without it, a query for "this patient's history" cannot distinguish information available before a given treatment decision from information recorded afterward, which is a prerequisite for any leakage-safe predictive model, not merely a data-hygiene preference.

2.3 Patient-Reported Outcome Measures in Orthopedics

Standardized PROMs are the primary mechanism by which orthopedic outcomes are quantified from the patient's own perspective. The Knee injury and Osteoarthritis Outcome Score (KOOS) and its short-form KOOS-JR were developed and validated by Roos and Lohmander for knee pathology [4]; the analogous Hip disability and Osteoarthritis Outcome Score (HOOS/HOOS-JR) was validated by Nilsdotter et al. [5]. Upper-limb function is captured by the Disabilities of the Arm, Shoulder and Hand (DASH) questionnaire and its abbreviated QuickDASH-11 form, developed by Beaton et al. specifically to reduce respondent burden while preserving measurement properties [6]. Spinal and cervical disability are measured respectively by the Oswestry Disability Index (ODI) [7] and the Neck Disability Index (NDI), validated by Vernon and Mior [8]. These instruments share a common scoring logic — summed or transformed item responses mapped onto a 0–100

disability or function scale — that the OPD AI Unit's scoring engine (Section 5.4) implements directly from each instrument's published manual rather than from a generic template.

2.4 Explainable Machine Learning on Tabular Clinical Data

Despite the prominence of deep learning, gradient-boosted decision trees remain the strongest default for structured, tabular data of the kind produced by an EHR-style system; Grinsztajn et al. show this empirically across a wide benchmark suite and attribute it to tree ensembles' native handling of heterogeneous, non-smooth, and missing feature distributions [9]. CatBoost specifically, introduced by Prokhorenkova et al., targets categorical-heavy tabular problems with an ordered boosting scheme that reduces target leakage during training and native categorical-feature handling without manual encoding [10] — both properties directly relevant to a dataset dominated by categorical fields such as body area, diagnosis, and treatment type. Post-hoc explainability for such models is commonly provided through SHAP (Shapley Additive Explanations), introduced by Lundberg and Lee as a unified, theoretically grounded feature-attribution framework [11]. Equally important, and frequently under-emphasized relative to discrimination metrics, is calibration: Van Calster et al. describe poor calibration as the "Achilles heel" of clinical predictive analytics, since a model can achieve strong ranking performance while still being systematically over- or under-confident in ways that mislead a clinician reading a single percentage [12]. Riley et al. further show that the sample size required to develop a reliable multivariable clinical prediction model is often far larger than intuition suggests, and propose formal minimum-sample calculations based on the number of candidate predictor parameters and expected outcome events [13] — a result this thesis applies directly in Section 7.9.

2.5 Clinical Speech Recognition and Ambient Documentation

Automatic speech recognition has matured to the point of practical use in clinical dictation; OpenAI's Whisper family of models, described by Radford et al., demonstrated that large-scale weakly supervised training could produce transcription robust enough for noisy, accented, multilingual real-world speech without domain-specific fine-tuning [14]. The broader "ambient documentation" trend — using speech and language models to convert a clinician's spoken encounter narrative directly into structured notes — is motivated by well-documented physician time lost to manual charting. The OPD AI Unit's dictation pipeline (Section 5.5) draws on this trend but deliberately departs from a pure end-to-end LLM approach for its safety-critical fixed-vocabulary fields, for reasons detailed in that section.

2.6 Mobile Health Applications in Orthopedic Follow-Up

Systematic reviews of mHealth interventions report consistent gains in patient engagement, appointment adherence, and self-reported outcome completion when patients are given direct, low-friction access to their own health information on a device they already carry [15]. Orthopedic follow-up is a particularly favorable use case because much of what needs to be tracked — pain, function, PROM trend over time — is naturally patient-reported and time-series in nature, which is exactly what the companion mobile app's Journey screen (Section 6.4) is designed to surface.

2.7 Research Gap

Much of the CDSS and clinical-ML literature falls into one of two categories: papers that propose and evaluate a predictive model on a static, often public or retrospective dataset with no connection to a live clinical workflow, or engineering

case studies of deployed clinical systems that do not attempt a rigorous, leakage-aware, auditable machine-learning layer. Comparatively little published work documents both halves of the problem being solved together, end to end, inside a single actively used system, with the predictive layer deliberately gated behind an explicit, literature-grounded readiness threshold rather than shipped as soon as a model file exists. This is the gap the OPD AI Unit occupies, and the honest, still-pending state of its predictive layer (Chapter 7) is itself evidence of that gating discipline in practice.

CHAPTER 3

Research Methodology

3.1 Research Paradigm: Design Science Research

This thesis adopts *Design Science Research* (DSR) as its governing paradigm. Hevner, March, Park, and Ram argue that information-systems research aimed at producing a novel, useful artifact for a real organizational problem — as opposed to purely explaining or predicting a naturally occurring phenomenon — is best evaluated against its own distinct standards: relevance to a genuine business or clinical need, rigor in construction and evaluation, and a demonstrable, communicable contribution to the knowledge base [16]. This fits the present work precisely: the artifact is a deployed clinical system solving a documented workflow and data-architecture problem for a real hospital department, and the research contribution is inseparable from the artifact's design decisions (the encounter boundary, the leakage-safe feature pipeline, the safe-refusal API contract) rather than a hypothesis tested in isolation from them.

3.2 Design Science Research Methodology (DSRM) Process

The project follows the six-activity DSRM process model of Peffers, Tuunanen, Rothenberger, and Chatterjee [17], entered at the *problem-centered* initiation point. Table 3.1 maps each DSRM activity to concrete project work.

Table 3.1 — DSRM activities mapped to project execution

DSRM Activity	Project execution
1. Problem identification & motivation	Workflow shadowing at the host department; formal data-access request; documented limitations of the patient-identifier-only data structure (Section 1.2).
2. Objectives of a solution	Primary and secondary objectives defined in Section 1.5, including explicit non-goals (no autonomous prescribing, no hidden confidence).
3. Design & development	Iterative build of the backend, frontend, dictation pipeline, encounter model, research export, offline trainer, and mobile companion (Chapters 5–7).
4. Demonstration	Live pilot deployment at the host department (Render-hosted backend, patient QR-code onboarding) and a second, independently built mobile client consuming the same API.
5. Evaluation	Unit-level scoring verification, API/access-control testing, and pipeline safety-guard testing (Chapter 8); predictive-model evaluation explicitly deferred and specified pending data sufficiency (Chapter 7).
6. Communication	This thesis, the in-repository research documentation (`docs/research-steps.md`), and the defense presentation.

3.3 System Development Methodology

Development followed an iterative, incrementally shippable process consistent with agile practice, evidenced by the project's own version-control history: an initial voice-recording prototype was followed by successive, independently usable increments — multilingual speaker distinction, a first working backend with patient creation, one-take structured dictation, a full UI redesign, pre-visit PROM categories, the encounter data model, the AI assistant scaffold, the research export,

and finally the mobile companion application. Each increment was integrated into the live pilot rather than accumulated in a long-lived branch. A deliberate methodological feature of this process is *staged, safety-gated rollout*: the AI recommendation module was shipped into production behind an explicit "unavailable" contract (Section 7.7) before any model existed, so that the feature could be integrated, wired into the UI, and audited end-to-end without ever being able to affect a real clinical decision prematurely — a pattern comparable to a feature-flagged dark launch in industry practice, applied here for clinical-safety rather than purely operational reasons.

3.4 Requirements Engineering Approach

Requirements were elicited primarily through direct engagement with clinical stakeholders in their existing roles — nurses completing pre-visit assessments, clerks managing patient intake and task queues, physicians and surgeons conducting evaluations — rather than through a generic feature backlog. The formal written request for access to the department's patient database, submitted before development began, functioned as both a governance gate and an early requirements artifact, since it required the project to specify in advance what data would be touched and for what purpose. Requirements were then refined iteratively: each new module (surgery evaluation, PROM assignment, follow-up calls, in-app messaging) was added in response to a workflow step observed to be missing rather than specified up front in full.

3.5 Data Collection Methodology for the Machine-Learning Track

The research track layered on top of the operational system follows a distinct, stricter data-handling methodology, summarized here and detailed in Chapter 7:

- **Unit of analysis:** one row per *encounter*, not one row per patient, so that repeated visits contribute independent, correctly time-ordered observations.
- **De-identification:** research exports exclude patient name, medical record number, phone, email, and address, substituting a salted, non-reversible pseudonymous hash for patient linkage across rows.
- **Chronological ordering:** encounters are sorted by date before any split is performed, so that training data always precedes test data in time.
- **Patient-level and time-based splitting:** no single patient's encounters are allowed to appear in both a training and a test partition, and the test partition is drawn from the most recent time period, matching the recommended protocol for clinical prediction-model evaluation.
- **Explicit leakage boundary:** only information available strictly before the treatment decision is eligible as a model input (Table 7.1); the treatment itself, follow-up outcomes, and any information recorded after the decision are structurally excluded from the feature set, not merely discouraged by convention.

3.6 Model Development Methodology

Model complexity is increased only where justified by measured improvement, following the standard statistical-learning principle that a more expressive model is preferred over a simpler one only when it earns that complexity on held-out data [18]. The planned comparison therefore proceeds in a fixed order — majority-class baseline, regularized logistic regression, random forest, CatBoost — so that any final claim about CatBoost's value is made relative to every simpler alternative, not in isolation.

3.7 Evaluation Methodology

Evaluation is planned as mixed-methods, combining quantitative model metrics (macro-F1, per-class precision/recall, top-3 recall, AUROC where applicable, calibration curves and Brier score, and abstention rate) with qualitative clinician feedback captured directly through the product's own `ModelFeedback` audit mechanism (accept / edit / reject / insufficient information). This mixed design follows the Framework for Evaluation in Design Science Research (FEDS) proposed by Venable, Pries-Heje, and Baskerville, which explicitly recommends combining artificial (offline, dataset-level) and naturalistic (real-user, in-context) evaluation episodes rather than relying on either alone [19]; the offline metrics correspond to the artificial evaluation episode, and the clinician-feedback audit trail is designed to support the naturalistic episode once the assistant reaches silent-mode deployment (Section 7.9).

3.8 Ethical Considerations

The project treats the research track as subject to the same ethical obligations as any clinical data study: institutional ethics/IRB review where applicable, explicit documentation of consent and permitted secondary use, role-based access control restricting who can view identifiable records, encrypted transport, and a hard rule against exporting direct identifiers for research purposes. This posture is consistent with the World Health Organization's guidance on the ethics and governance of artificial intelligence for health, which emphasizes human oversight, transparency, and accountability as preconditions for trustworthy clinical AI [20]. Concretely, the research hash salt that pseudonymizes patient identity in exports is required to be stored outside source control and never left at its development default (Section 7.5) — a governance control, not merely a configuration note.

3.9 Technology Selection Methodology

Technology choices were made to minimize the distance between a working prototype and a production-safe pilot deployment rather than to showcase novelty. FastAPI was chosen for the backend for its native request/response type validation and dependency-injected authentication, which made role-based access control (Section 4.8) straightforward to enforce consistently across twenty-one routers. React with Vite was chosen for the clinical frontend for fast iteration during weekly stakeholder demonstrations. CatBoost was chosen for the predictive layer specifically because the target dataset is small-to-medium, categorical-heavy, and contains missing values by nature of real clinical documentation — exactly the regime in which CatBoost's ordered boosting and native categorical handling are strongest [10] — rather than because it is currently fashionable; the methodology explicitly required this choice to be justified against simpler baselines (Section 3.6), not asserted. Expo/React Native was chosen for the mobile companion so a single TypeScript codebase could target both iOS and Android without duplicating the authentication and data-fetching logic already proven on the web platform.

CHAPTER 4

System Analysis and Design

4.1 As-Is Workflow Analysis

Prior to this system, a typical patient pathway at the host department ran through paper or ad hoc digital forms for intake, a manually calculated (or uncalculated) pre-visit score, a physician consultation with no structured summary of prior visits readily at hand, hand-written or loosely structured diagnosis and treatment notes, and a follow-up process dependent on staff remembering to call. Each step was individually manageable but nothing connected them: a physician could not easily see a patient's PROM trend across visits, a nurse could not see whether a diagnostic ordered last visit had ever been completed, and a patient had no way to review their own diagnosis or plan outside the clinic walls.

4.2 Stakeholder Roles

Table 4.1 — System actors and primary responsibilities

Role	Primary system responsibilities
Clerk	Patient registration, task queues, pre-visit PROM assignment and clerk-assisted completion, document intake.
Nurse	Nurse-assisted PROM assessment, vitals/pre-visit data capture, patient status tracking; explicitly barred from surgery-evaluation routes.
Physician / Surgeon	Physician evaluation, dictation, diagnosis, diagnostic ordering, treatment recording, surgery evaluation, AI-assistant review and feedback, "Send to Patient" reporting.
Admin	Assessment/status configuration, follow-up settings, staff account management.
Patient	Self-service PROM completion, mobile visit-history review, document access, clinic messaging — authenticated separately from staff via MRN + one-time password.

4.3 Functional Requirements Overview

Functional requirements group naturally around the visit pathway: patient registration and record management; bilingual pre-visit PROM assessment with multiple completion sources (patient self-fill, nurse-assisted, clerk-assisted, physician-assisted); physician evaluation with AI-assisted dictation; diagnostic ordering and result tracking; treatment recording and outcome capture; document upload and patient-facing retrieval; follow-up call scheduling and notification; in-app staff–patient messaging; analytics and de-identified research export; and the AI clinical-assistant panel described fully in Chapter 7.

4.4 Non-Functional Requirements

- **Bilingual, bidirectional UI** — full Arabic (RTL) and English (LTR) coverage across every clinical and patient-facing screen, with per-page locale bundles rather than a single monolithic translation file.
- **Role-scoped security** — JWT-based staff authentication with per-role route dependencies, and an *audience-separated* patient JWT so a patient token can never be replayed against a staff-only route.

- **Auditability** — every AI suggestion and every clinician response to it is persisted, never discarded after display.
- **Low-friction patient access** — one-time-password login by medical record number, requiring no password to remember, with front-desk-assisted QR-code onboarding for first use.
- **Graceful AI degradation** — every AI-dependent feature (dictation extraction, patient summaries, treatment recommendation) must fail safely to a usable non-AI fallback rather than blocking the clinical task.
- **Client/logic reuse** — the mobile client must consume the same backend and, where practical, the same computation (e.g., PROM trend logic) as the web client, rather than re-implement it.
- **Cloud deployability** — the backend must run as a stateless web service against an externally configured PostgreSQL database in production (Render.com), with SQLite retained only for local development.

4.5 System Architecture

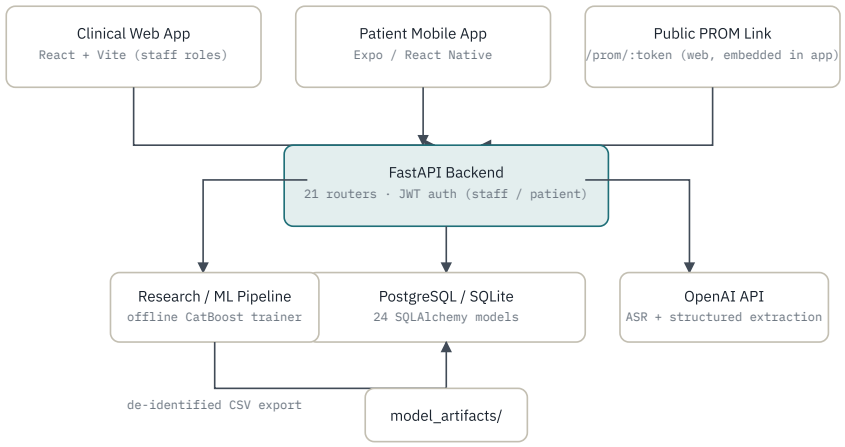
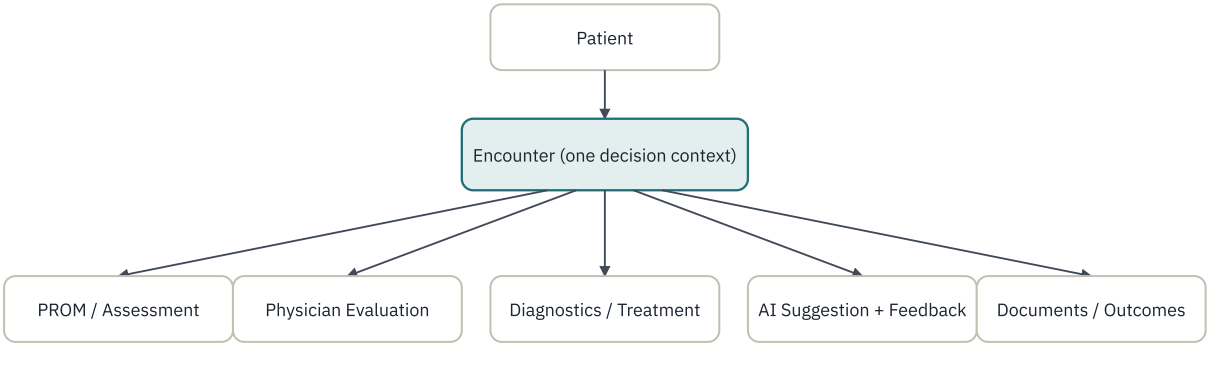


Figure 4.1 — System architecture. Two independent clients (web and mobile) and one embedded public form share a single FastAPI backend and database; the OpenAI API and the offline ML pipeline are separate concerns attached at the backend boundary, neither one able to bypass the same authentication and audit layer the clinical routes use.

4.6 Encounter-Centered Data Model



Every clinical record below the encounter line carries an encounter_id, giving the pre-/post-decision boundary Chapter 7 relies on.

Figure 4.2 — The Encounter entity is the pivot between patient-level identity and visit-level clinical data. All downstream records — assessment, evaluation, diagnostics, treatments, AI suggestions and feedback, documents, and later outcomes — are linked to one encounter, which is what allows the system to know exactly what was known before a treatment decision was made.

4.7 Security Architecture

Two independent JWT audiences are issued: a staff token (role-carrying, used by `get_current_user` and the per-role `forbid_roles` dependency) and a patient token (issued only after OTP verification, used by `get_current_patient`). The two are not interchangeable by design, so a leaked patient session cannot be replayed against any staff route, and vice versa. Patient one-time passwords are bcrypt-hashed before storage, expire after five minutes, lock out after five failed attempts, and — in the absence of a configured SMS provider — fall back to a development-mode path that prints the code to the OTP screen and backend console rather than silently failing, a deliberate choice that keeps the pilot usable before an SMS vendor contract is in place while keeping that fallback explicitly gated behind an environment flag (`OTP_DEV_MODE`) that must be turned off for a real deployment. Surgery-evaluation routes are further restricted with `forbid_roles("nurse")`, and the de-identified research export is gated behind staff authentication like every other protected router.

CHAPTER 5

Implementation — Web Platform

5.1 Technology Stack

Table 5.1 — Web platform technology stack

Layer	Technology	Purpose
API framework	FastAPI + Uvicorn	Typed request/response models, dependency-injected auth
ORM / database	SQLAlchemy; PostgreSQL (prod, `psycopg2-binary`) / SQLite (dev)	24 models across clinical, AI-audit, and communication domains
Auth	`pyjwt`, `passlib[bcrypt]`, `bcrypt`	Staff JWT, patient JWT, OTP hashing
Audio processing	`pydub`, `audioop-lts`	Dictation audio handling (Python 3.13 stdlib backport)
AI services	`openai` SDK — `gpt-4o-mini-transcribe`, `gpt-4o-mini`	Speech transcription, structured extraction, patient summaries
ML research stack	`pandas`, `catboost`, `scikit-learn`	Treatment-choice model training (separate `requirements-research.txt`)
Frontend	React 18 + Vite, `react-icons`	Clinical SPA, custom i18n, light/dark theming
Deployment	Render.com web service (`render.yaml`)	Cloud hosting, environment-scoped secrets

5.2 Backend Architecture

The backend is a single FastAPI application (`backend/main.py`) composing twenty-one routers, each mounted with an explicit dependency chain rather than a global middleware, so that authentication and role restrictions are visible at the point of registration: `app.include_router(surgery_evaluations_router, dependencies=[Depends(forbid_roles("nurse"))])` is legible on its own without cross-referencing a separate policy file. The twenty-four SQLAlchemy models fall into five functional groups: core clinical records (`Patient`, `Encounter`, `Assessment`, `Evaluation`, `SurgeryEvaluation`, `Diagnostic`, `Treatment`, `TreatmentOutcome`, `Document`, `InjuryEvent`); configuration/admin (`AssessmentConfig`, `StatusConfig`, `FollowUpSettings`, `DiagnosticTestOption`, `TreatmentOption`); scheduling and outreach (`FollowUpCall`, `PROMAssignment`); AI/research audit (`ModelSuggestion`, `ModelFeedback`); and patient-facing/communication (`User`, `PatientOTP`, `PatientNotification`, `StaffNotification`, `ChatMessage`). Schema evolution favored additive, backward-compatible fields — the encounter linkage on legacy tables, for instance, was added as a nullable foreign key rather than a breaking migration, so historical records remain valid while new records opt into the stricter encounter boundary.

5.3 Frontend Architecture

The clinical frontend is a React single-page application organized around role-oriented pages (`AllPatients` , `PatientProfile` , `PatientStatus` , `PreVisitAssessment` , `PhysicianEvaluation` , `SurgeryEvaluation` , `Surgeries` , `NurseDashboard` , `ClerkTasks` , `DocumentGenerator` , `Analytics` , `Messages`) inside a shared `DashboardLayout` . Cross-cutting concerns are extracted into hooks: `useDictation` (recording state machine and live-caption polling), `useLanguage` (Arabic/English switching with RTL layout flip), `useLookupData` (shared diagnostic-test and treatment-option vocabularies also consumed server-side by the dictation extractor), `usePatients` , and `useTheme` (light/dark, mirrored token-for-token in the mobile app's own theme file). Internationalization is organized per page rather than as one monolithic dictionary — thirteen matched JSON bundles exist under `i18n/locales/ar` and `i18n/locales/en` — which keeps translation review scoped to the screen being changed.

5.4 Pre-Visit PROM Assessment Module

The clinic's pre-visit assessment step is standardized around six validated instruments selected to cover the department's full body-area range, sourced and scored directly from their original publication manuals rather than a generic template (the `scores/` reference set in the repository includes the KOOS manual, the HOOS hip survey with an Arabic translation, and the neck/NDI scoring notes used to implement the corresponding scoring functions).

Table 5.2 — PROM instruments implemented in `scoring.js`

Instrument	Body area	Scoring approach	Direction
KOOS-JR	Knee	Non-linear raw-to-interval conversion (validated official table)	0 = worst, 100 = best
HOOS-JR	Hip	Non-linear raw-to-interval conversion (validated official table)	0 = worst, 100 = best
QuickDASH-11	Wrist / hand / elbow / shoulder	`((sum/n) − 1) × 25` , requires ≥10 of 11 items answered	0 = best, 100 = worst
ODI	Lower back	`(total / (answered × 5)) × 100`	0 = best, 100 = worst
NDI	Neck	`(total / (answered × 5)) × 100`	0 = best, 100 = worst
SEFAS	Foot / ankle	Summed item score, standardized 0–100 scale	0 = worst, 100 = best

Each PROM can be completed through four distinct sources tracked on the record itself — patient self-fill (via a tokenized public link, `prom_public` router, also embedded inside the mobile app's WebView), nurse-assisted, clerk-assisted, or physician-assisted — and the `PROMAssignment` model tracks which is expected for a given encounter, which is what lets the mobile app correctly show a "Start" button only for self-fill assignments and an informational-only state for staff-assisted ones. Longitudinal score trends are computed once, server-side, by `prom_trend.py` , and reused unmodified by both the staff dashboard's `PromTrendChart` and the mobile app's Journey screen — a concrete instance of the reuse-over-duplication principle applied throughout the system.

5.5 AI-Assisted Clinical Dictation

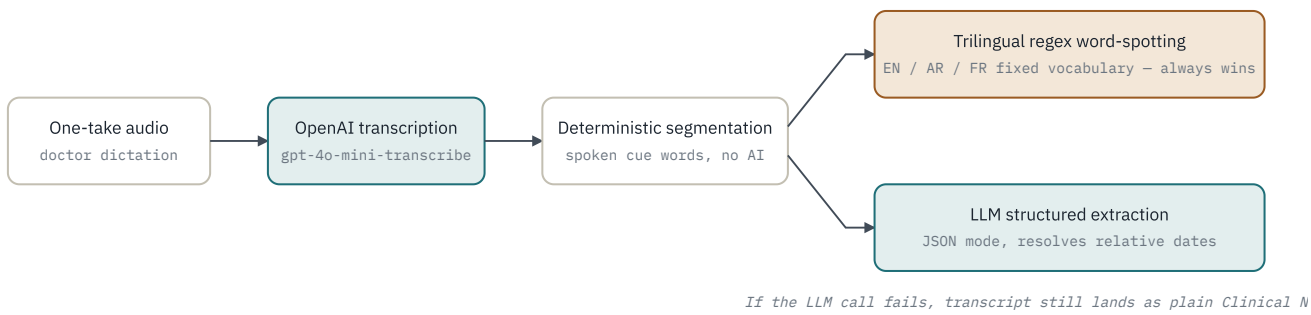


Figure 5.1 — The "Record Full Visit" dictation pipeline (`backend/llm_extract.py`). Deterministic segmentation and multilingual keyword-matching run independently of the LLM call, so a fixed-vocabulary field (a diagnostic test, a treatment option) is always resolved by the clinician's own spoken word, never overwritten by a model's paraphrase of it.

The "Record Full Visit" feature lets a physician dictate diagnosis, diagnostics, and treatment plan as a single continuous recording rather than filling three separate fields by hand. A live caption preview fires roughly every four seconds during recording so the doctor can see the transcript forming in near real time. Once recording stops, `segment_dictation()` splits the transcript into evaluation, diagnostics, and treatment chunks using spoken cue words ("diagnosis…", "diagnostics…" / "order tests…" / "labs…", "treatment plan…") — a deterministic, no-AI step chosen specifically so the split is predictable and doctors can learn the cues, rather than depending on a language model to decide, non-deterministically, where one section ends and the next begins. In parallel, a fixed trilingual regex vocabulary (English, Arabic with normalization for hamza and alternate ta-marbuta forms, and French) matches the five diagnostic-test and six treatment-option categories directly against the raw transcript. Only after this deterministic pass does `extract_structured()` send the transcript and chunk boundaries to OpenAI in JSON mode to map remaining free speech onto the same fixed vocabularies and resolve relative spoken dates ("in two weeks") into ISO dates. If the OpenAI call fails for any reason, the pipeline degrades gracefully: the plain segmented transcript is still written into Clinical Notes, and the physician completes the structured diagnostic-test and treatment-plan fields manually, so a transient API outage never blocks documentation of a visit.

5.6 AI-Generated Patient Visit Summaries

A second, independent language-model feature generates a short, plain-language summary of a visit for the patient themselves, shown in the mobile app's report detail screen and included in its PDF export. The prompt is constrained explicitly to the diagnosis, assessment, and plan already on file, with an instruction never to invent, assume, or add any detail not stated, and the model is directed to write directly to the patient in a warm, non-alarming register and briefly explain any clinical term it must use. Generation happens exactly once, at the moment a physician or surgeon marks a report "sent to patient," and the result is stored permanently on that record rather than regenerated on each read — so what the patient sees always matches what was true the moment their doctor released it, even if the underlying evaluation is later amended. Failure is silent and non-blocking: if the OpenAI call errors, the summary is simply omitted and the "Send to Patient" action itself still succeeds.

5.7 Diagnostics, Treatments, Documents, and Surgery Evaluation

Diagnostic orders and results, treatment records, and uploaded documents are each modeled as their own entity linked to an encounter where one exists, with a dedicated surgery-evaluation workflow (role-restricted to non-nurse staff) covering the operative-specific documentation a general evaluation does not capture. The `PrintDocModal` and `PrintTypeModal` components generate patient-facing printable documents directly from the same structured records the physician entered, removing a second manual transcription step.

5.8 Follow-Up, Notifications, and Messaging

Scheduled follow-up is modeled through `FollowUpSettings` (protocol-level windows) and `FollowUpCall` (individual outreach attempts), surfaced on the clerk dashboard as a horizontally scrollable due-calls list. Two parallel notification tables — `StaffNotification` and `PatientNotification` — drive in-app inboxes on each side; a patient notification row is created the moment a report is sent or a new chat message arrives, which is also the exact point at which a real push notification would be dispatched once device push tokens are wired up (Section 6.7). Messaging itself runs through one shared `ChatMessage` table read by both the web dashboard's Messages page (where any logged-in nurse or physician can open a patient thread and reply) and the mobile app; the patient always sees replies simply as "Care Assistant," never an individual staff member's name, which keeps the clinical team fully interchangeable behind a single consistent patient-facing identity.

5.9 Analytics and Research Export

The `Analytics` page gives staff an operational view of clinic activity, while the protected `GET /api/research/encounters.csv` endpoint produces the de-identified, one-row-per-encounter export that feeds the offline model-training pipeline described in Chapter 7 — the same authentication and role protection used everywhere else in the system, not a separate, weaker export path.

5.10 Deployment

The backend deploys to Render.com as a Python web service (`render.yaml`), building from `requirements.txt` and starting via Uvicorn bound to the platform-assigned port. Secrets — the database URL, the OpenAI API key, and the transcription/extraction model identifiers — are injected as environment variables rather than committed to source, and the OTP development fallback is controlled by its own explicit flag so it can be confirmed off before any real deployment is treated as production-ready.

5.11 System Evolution

Table 5.3 — Web platform development milestones (paraphrased from project version history)

Phase	Period	Milestone
0	Mar 2026	Initial prototype: voice recording, waveform playback, multilingual voice/speaker distinction
1	Apr–May 2026	Speaker-recognition refinement; project skeleton and demo scope defined
2	Jun 2026	Full FastAPI backend introduced, including patient creation
3	Jun–Jul 2026	One-take structured dictation (diagnosis / diagnostics / treatment), UI redesign, pre-visit PROM categories

Phase	Period	Milestone
4	Aug 2026	Encounter-centered data model, AI clinical-assistant scaffold, de-identified research export; mobile companion app begun
5	Sep 2026	Mobile app wired to live data (messaging, notifications); bilingual polish; assessment/imaging results module

CHAPTER 6

Implementation — Mobile Companion Application

6.1 Rationale for a Separate Codebase

The patient-facing mobile app (`opd-mobile-app`) is deliberately maintained as a separate repository from the web platform. Mobile release cycles, native build tooling, and app-store review operate on a different rhythm than web/backend iteration, and decoupling the two means neither blocks the other — a backend fix can ship the same day it is written, while a mobile release can follow its own build-and-review cadence without holding up clinical operations.

6.2 Technology Stack

The app is built with Expo SDK 57 and `expo-router` file-based routing, in strict TypeScript, on React Native 0.86 / React 19. `axios` handles API calls; the session token is stored via `expo-secure-store` on native platforms and `localStorage` on web; `expo-file-system` and `expo-sharing` support authenticated document download and viewing; and `react-native-webview` embeds the web app's own PROM fill-in page rather than reimplementing its form and scoring logic natively.

6.3 Authentication Flow

A patient enters their medical record number, which the backend resolves to a phone number on file and sends a one-time password to (or, in development mode, displays directly, per Section 4.7); verifying that code returns a patient-scoped JWT distinct from the staff token, stored via `tokenStorage.ts` , and gates the tab navigator behind `Stack.Protected` so an unauthenticated user can reach only the MRN and OTP screens.

6.4 Information Architecture and Screens

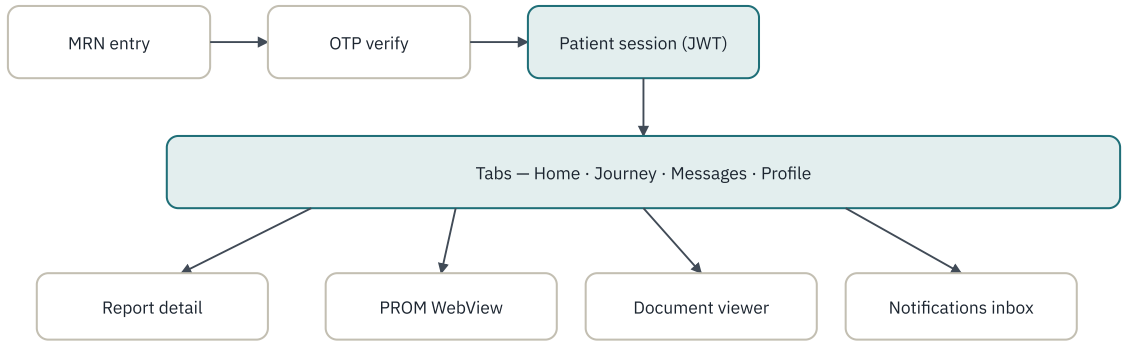


Figure 6.1 — Mobile app information architecture. Four tabs sit behind MRN + OTP authentication; report, PROM, document, and notification screens are reached from Home or Journey rather than duplicated as their own tabs.

Table 6.1 — Mobile app screen inventory

Screen	Backed by	Notes

Screen	Backed by	Notes
Home	`/patient/appointment`, `/patient/active-care`, `/patient/prom-status`	Next appointment, PROM reminder, active care plan, notification bell
Journey	`/patient/prom-trend`, `/patient/visit-reports`	Reuses backend's `compute_prom_trend`, same logic as staff dashboard
Messages	`/patient/messages`	Real two-way chat, 8-second poll, shown as "Care Assistant"
Profile	`/patient/me`	Real identity; notification/language/appearance toggles are local-only for now
Report detail	`/patient/visit-reports/{id}`	SOAP sections, AI plain-language summary, PDF export via `expo-print`
Document viewer	`/patient/documents/{id}/file`	Full-screen images in-app; PDFs via system share sheet
Notifications	`/patient/notifications`	One row per "report sent" or chat event
PROM fill-in	WebView of web app's `/prom/:token`	Self-fill only; staff-assisted PROMs show informational-only

6.5 Design System Consistency

`theme/colors.ts` is a direct, token-for-token port of the web app's blue-turquoise palette, kept manually in sync rather than shared via a package, and dark mode follows the device's system setting automatically through `useColorScheme()` — the same three-state theming discipline (light / dark / system) this thesis report itself follows.

6.6 Cross-Platform Backend Reuse

Every screen in the mobile app reads live data from the same backend the web platform uses; there is no mock-data or sample-data module anywhere in the project. This is a meaningful validation in its own right: a second, independently built client, written in a different language (TypeScript/React Native rather than JavaScript/React) by the same small team, was able to consume the existing API surface without needing new endpoints beyond the already-present `patient_self.py` router — evidence that the API design generalizes rather than having been implicitly shaped around the web client's specific needs.

6.7 Known Limitations and Deferred Features

Push notifications are in-app only at present. Real device push through the Expo push service is deferred because Expo Go cannot receive remote push on Android as of SDK 53, which would require an `expo-dev-client` build to resolve; the server-side trigger point already exists in `backend/notifications.py`, so wiring up real push later is additive rather than a redesign. Profile preferences (notification toggles, language, appearance) remain local-only UI state, since the backend does not yet have a per-patient preference store to back them.

6.8 Development Timeline

The mobile app's own history spans roughly four weeks: an initial commit on 16 August 2026, a first working scaffold of tabs, screens, and theme by 26 August, full wiring to live data — messaging, notifications, and other services — the following day, and design and language polish through 8–11 September. Built directly against the already-mature backend documented in Chapter 5, this compressed timeline is itself a demonstration-activity result in the DSRM sense (Section 3.2): the encounter-centered platform was mature enough to support a second client with minimal new backend work.

CHAPTER 7

The AI Clinical Decision-Support Module

This chapter documents the design, implementation, and current status of the treatment-recommendation module — the part of the system most directly tied to the thesis's primary research objective — and reports its results and limitations without overstatement.

7.1 Design Philosophy

The module is governed by a small set of non-negotiable rules, enforced in code rather than left as policy: the assistant never automatically prescribes, never automatically orders a diagnostic test, never automatically saves a treatment without explicit clinician confirmation, never hides a low-confidence result, and never fabricates a suggestion when no validated model is available — it reports an explicit unavailable state instead (Section 7.7). Every suggestion the assistant ever produces is stored permanently alongside the clinician's response to it, so the system's influence on real decisions is always reconstructable after the fact.

> **SAFETY FRAMING**
>
> This module is decision support, not an autonomous medical device. The physician remains the sole author of the final treatment decision at every point in this pipeline.

7.2 The Encounter as the Unit of Analysis

As established in Chapter 4, the `Encounter` entity is what makes this module possible at all. Before it existed, records were linked only by patient identifier, so a query for "this patient's history" could not distinguish information available before a specific treatment decision from information recorded afterward — precisely the condition under which a model can silently learn from the future. The encounter closes that gap by giving every downstream record (assessment, evaluation, diagnostics, treatment, AI suggestion, outcome) a single shared decision context to attach to.

7.3 Feature Engineering and the Leakage Boundary

The model's feature set (`FEATURES` in both `backend/routers/ai_assistant.py` and `backend/research/train_treatment_model.py`, kept identical between training and inference on purpose) is limited to thirteen fields, every one of which is knowable before a treatment decision is made.

Table 7.1 — Treatment-choice model feature set and leakage classification

Feature	Type	Known before treatment decision?
`body_area`	categorical	yes
`encounter_type`	categorical	yes
`previsit_source`	categorical	yes
`gender`	categorical	yes

Feature	Type	Known before treatment decision?
`prom_code`	categorical	yes
`age`	numeric	yes
`prom_score` / `prom_max_score` / `final_score`	numeric	yes
`pain_score`	numeric	yes
`diagnostic_types`	categorical (multi-value)	yes — pre-existing only
`chief_complaint`	categorical	yes
`diagnosis`	categorical	yes

Structurally excluded: the treatment being predicted, follow-up PROM score, outcome response, later diagnosis changes, later complications, future escalation.

Two implementation details enforce this boundary rather than merely documenting it. First, both the assessment and evaluation records used to build a feature row are filtered to the same encounter (or to legacy null-encounter records) before being read, so a value from a different, later visit can never silently enter the feature vector. Second, the diagnostics considered are scoped identically — encounter-matched diagnostics only — a correction applied specifically because an unscoped query would otherwise feed every diagnostic the patient has ever had, including from future or unrelated encounters, into a prediction for "today," producing a feature distribution the model was never trained on. Enforcing the same scoping rule at both training-data export time and inference time is what keeps the two paths honest with each other.

7.4 Treatment Taxonomy and Multi-Label Target

Free-text treatment records are normalized into six stable categories — medication, physiotherapy, injection therapy, surgery, rest and monitoring, and long-term rehabilitation — while the clinician's original text is preserved separately and never overwritten. Because a single encounter can conclude with more than one concurrent treatment (for example, medication alongside physiotherapy), the target is modeled as multi-label rather than single-class, using scikit-learn's `MultiLabelBinarizer` to convert the pipe-delimited label set into a binary indicator matrix before fitting.

7.5 De-Identified Research Data Pipeline

The protected `GET /api/research/encounters.csv` endpoint produces exactly one row per encounter, containing a salted pseudonymous patient hash, encounter fields, demographics, PROM features, pain score, diagnosis, diagnostic types, treatment labels, and — where available — the latest linked follow-up score. Direct identifiers (name, MRN, phone, email, address) are excluded entirely rather than masked. The salt used to generate the pseudonymous hash is read from the `RESEARCH_HASH_SALT` environment variable and must never be left at its development default for a real research export — a governance control the export code itself enforces by convention, and one this thesis flags again as a deployment prerequisite in Section 7.9.

7.6 Model Development Pipeline



Figure 7.1 — The intended hybrid AI pipeline. The structured CatBoost ranking model is responsible for treatment ranking; a language model, where used elsewhere in the system, is confined to explaining information, never to deciding it.

The offline trainer (`backend/research/train_treatment_model.py`) loads the encounter CSV, sorts rows chronologically, and refuses outright to proceed if fewer than thirty labeled encounters or fewer than two distinct treatment classes are present — a hard-coded safeguard, not a suggestion, raising a `SystemExit` rather than producing a degenerate model. Where it does proceed, it applies an 80/20 chronological split, treats age/PROM/pain fields as numeric and every other feature as categorical, and fits a `CatBoostClassifier` configured for multi-label output (`loss_function="MultiLogloss"`, 250 iterations, depth 5, learning rate 0.05, fixed random seed 42) against the `MultiLabelBinarizer`-encoded target. When the CatBoost/pandas/scikit-learn research dependencies are not installed, the script still runs and writes a metadata artifact marked `status: "validated_only"` rather than failing silently — the artifact schema itself carries a permanent `warning` field stating that it predicts observed clinician treatment choice, not causal treatment benefit, so that warning travels with the model file wherever it is copied.

7.7 Safe-Serving Architecture

Two protected endpoints expose the model: `GET /api/ai/status` and `POST /api/ai/patients/{patient_id}/recommendation`. Both check, before doing anything else, whether a metadata file and a matching `.cbm` model file exist on disk and whether that metadata is marked `trained`; if not, they return `available: false` with an explanatory message rather than any suggestion. When a validated artifact is present, the recommendation endpoint assembles the same thirteen-feature snapshot described in Section 7.3 for the patient's current (or specified) encounter, runs it through the loaded CatBoost model, ranks the top three predicted treatments by probability, and — regardless of outcome — writes a `ModelSuggestion` row recording the model version, the exact feature snapshot used, the ranked suggestions, any warnings (for example, a missing pre-visit assessment), and a timestamp. On the Physician Evaluation screen, this surfaces as an "AI Clinical Assistant" panel carrying an explicit "decision support only" disclaimer, a model-version label, and four response actions — accept, edit in the treatment plan, reject, or mark "not enough information" — each of which is persisted to a `ModelFeedback` row through `POST /api/model-suggestions/{suggestion_id}/feedback`.

7.8 Current Implementation Status and Preliminary Technical Validation

This section reports, without embellishment, exactly what has and has not been established at the time of writing.

Table 7.2 — AI module readiness by component

Component	Status	Evidence
Encounter data model	VERIFIED	In production schema; every new clinical record carries `encounter_id`
Leakage-safe feature scoping	VERIFIED	Encounter-scoped filtering confirmed identical at export and inference time (Section 7.3)
De-identified export	VERIFIED	`GET /api/research/encounters.csv` live and access-controlled
Minimum-sample & class-diversity refusal guard	VERIFIED	Trainer raises `SystemExit` below 30 rows or <2 classes
CatBoost multi-label fitting path	VERIFIED	Runs correctly against the research dependency stack; artifact serialization confirmed
Safe-unavailable API contract	VERIFIED IN PRODUCTION	`GET /api/ai/status` currently returns `available: false`, as designed
Audit trail (`ModelSuggestion` / `ModelFeedback`)	VERIFIED	Schema and protected routes in production, wired into the Physician Evaluation UI
Clinically trained, deployed model artifact	PENDING	No `model_artifacts/treatment-choice.cbm` promoted to production yet
Predictive performance metrics (macro-F1, calibration, subgroup)	PENDING	Not yet measurable — no clinically reviewed dataset has crossed the sufficiency threshold

In concrete terms: the entire mechanical pipeline — from a raw encounter export through a leakage-safe split, a refusal safeguard, a CatBoost fit, artifact serialization, and a safe-unavailable inference contract — has been exercised and behaves as designed. What has *not* happened is training on a clinically representative, clinician-reviewed dataset large enough to trust. The encounter volume accumulated since the encounter model shipped, within the March–September 2026 pilot window documented in Table 5.3, has not yet reliably and verifiably crossed even the pipeline's own engineering minimum of thirty labeled encounters with clinician-reviewed treatment labels across at least two classes — and that engineering minimum, as Section 7.9 argues, is itself far below what would be required for a clinically defensible model. The system therefore correctly serves `available: false` today.

> **WHY NO PERFORMANCE NUMBERS ARE REPORTED HERE**
>
> No accuracy, F1, AUROC, or calibration figures for the treatment-choice model appear anywhere in this thesis, because none exist yet on real, clinician-reviewed data. Reporting placeholder or simulated figures as if they were results would misrepresent the state of the work and would be indefensible at the sample sizes currently available. The project's own internal research documentation states this constraint explicitly, and this thesis honors it rather than working around it.

This null-but-validated outcome is itself treated here as a legitimate methodological finding, consistent with the DSR evaluation framing of Section 3.7: a system that correctly refuses to guess under insufficient evidence is direct evidence that its leakage-prevention and refusal-safeguard logic works as designed,

rather than a gap in the thesis. The contribution at this stage is a validated, reusable, safety-first reference architecture — not a claim of clinical predictive performance, which Section 7.9 turns instead into concrete, actionable guidance for the deployment phase that follows.

7.9 Suggestions for the Medical and Clinical Deployment Path

Because the predictive layer is not yet live, this section converts that gap into the thesis's most direct clinical contribution: a concrete, literature-grounded set of recommendations for what must happen before the AI Clinical Assistant panel is ever switched from disclaimer-only to a live suggestion source.

- **Recalibrate the sample-size target upward from the engineering minimum.** The trainer's thirty-row threshold exists only to prevent an ill-posed CatBoost fit, not to certify clinical adequacy. Riley et al.'s minimum-sample framework for multivariable clinical prediction models ties required sample size to the number of candidate predictor parameters and expected outcome events per class [13]; applied to this model's thirteen candidate features and six treatment classes, a defensible per-body-area pilot model plausibly requires several hundred clinician-reviewed encounters per body area, not thirty pooled across all of them. The department's six body areas should each be tracked toward this target independently.
- **Require a clinician-led label review pass before any training run.** Treatment-taxonomy mapping from free text to the six normalized categories should be signed off by an orthopedic attending, not inferred purely from automated matching, before that label is trusted as ground truth.
- **Train per-body-area models rather than one pooled model.** Different body areas plausibly have different treatment-decision dynamics; a single pooled model risks averaging away clinically meaningful differences between, for example, knee and spine decision-making.
- **Report calibration and Brier score alongside discrimination metrics as a hard requirement, not an optional extra.** Van Calster et al. identify poor calibration as the most common and most dangerous failure mode in clinical predictive analytics, precisely because a well-discriminating but poorly calibrated model can still mislead a physician reading a single confidence percentage [12].
- **Audit subgroup performance by age, gender, and body area before any deployment decision**, so that differential performance across patient groups is caught in an offline report rather than discovered in clinical use.
- **Run a formal silent-mode (shadow) validation period** — generating and logging suggestions without ever displaying them to physicians — comparing model output against actual clinician decisions over a defined window, before the UI panel is switched on for any real user.
- **Establish a standing model-card and re-training governance process**, versioning every promoted artifact and defining triggers (elapsed time, drift in observed treatment-class distribution) for re-training as practice patterns evolve.
- **Confirm institutional ethics/IRB sign-off and documented patient-consent language** before any research export leaves the controlled environment, and rotate `RESEARCH_HASH_SALT` to a real secret outside source control before that export is treated as a production research artifact (Section 7.5).

- **Operationalize standardized follow-up windows** (for example 3, 6, and 9 months) by tying `FollowUpCall` scheduling automatically to encounter type, so that `TreatmentOutcome` completeness — currently the limiting factor for any future outcome-effectiveness study, a separate and harder question from treatment-choice prediction — improves systematically rather than depending on staff memory.

7.10 Prediction of Practice vs. Proof of Efficacy

A final epistemic distinction must anchor every future claim made about this module. The treatment-choice model, once trained, predicts *which treatment a clinician selected* given the information available at the time — a prediction of observed practice. It does not, and without a materially different observational-confounding-aware or randomized study design cannot, predict *which treatment would have produced the best outcome* for that patient. Treatment selection in the source data is influenced by disease severity, patient preference, and other factors the model does not observe — a textbook case of confounding by indication. The acceptable claim for this thesis is that the system provides explainable predictions of observed clinician treatment choices from pre-decision orthopedic data; the unacceptable claim, which this thesis explicitly declines to make, is that the system determines the medically optimal treatment, now or once trained on the current pilot's data volume.

CHAPTER 8

Testing and Evaluation

8.1 Testing Strategy Overview

Testing spans four levels: unit-level verification of scoring and pipeline logic, integration-level verification of API access control, manual clinical-role testing during pilot use, and — specific to the AI module — direct verification of its safety-refusal behavior, which is executable and checkable today even though no trained model yet exists.

8.2 PROM Scoring Engine Verification

The scoring engine carries its own automated unit tests (`frontend/src/utils/scoring.test.mjs`), and each formula was implemented and cross-checked directly against its instrument's published scoring manual rather than a secondary summary: the KOOS and HOOS reference PDFs and the neck/NDI scoring notes included in the project's `scores/` reference directory were used question-by-question to verify the non-linear raw-to-interval conversions for KOOS-JR/HOOS-JR and the percentage formulas for ODI/NDI, including edge-case handling such as the QuickDASH-11's requirement that at least ten of eleven items be answered before a score is computed at all.

8.3 Dictation Pipeline Verification

The deterministic segmentation and trilingual keyword-matching layers (Section 5.5) were verified for cue-word coverage across English, Arabic (including hamza-form and definite-article normalization), and French phrasings of each diagnostic test and treatment option, and the graceful-degradation path — plain transcript delivered to Clinical Notes when the OpenAI extraction call fails — was exercised directly by simulating an API failure, confirming the physician-facing workflow never blocks on a third-party outage.

8.4 API and Access-Control Testing

Role-gated routing was verified by confirming that a nurse-scoped staff token is rejected by the `forbid_roles("nurse")`-protected surgery-evaluation routes, and that a patient-scoped JWT issued via OTP verification is rejected by every staff-only route and vice versa, matching the audience-separation design of Section 4.7. OTP behavior was verified for expiry (five minutes), attempt lockout (five failed attempts), and the development-mode console fallback path.

8.5 AI Assistant Safety-Contract Testing

The treatment-model pipeline's refusal guards were tested directly against their own stated thresholds: running the trainer against fewer than thirty labeled rows, and against a single-class label set, both correctly raise `SystemExit` rather than produce a degenerate model, and the live `GET /api/ai/status` and `POST /api/ai/patients/{id}/recommendation` endpoints were confirmed to return an explicit `available: false` response with a human-readable explanation when no validated artifact is present — the exact behavior the production system exhibits today, as reported in Table 7.2.

8.6 Cross-Client Consistency Testing

Because the mobile app's Journey screen reuses the backend's own `compute_prom_trend` function rather than reimplementing it, and its PROM

fill-in screen embeds the web app's own `/prom/:token` page rather than a second scoring implementation, cross-client consistency is structurally guaranteed rather than something that must be separately regression-tested on every scoring change — a design choice that functions as its own ongoing test strategy.

8.7 Usability Considerations

Bilingual layout correctness was checked across representative screens in both Arabic (right-to-left) and English (left-to-right) modes. The one-time-password login flow was chosen over a remembered password specifically for usability with a patient population that may not be highly digitally literate, and front-desk-assisted onboarding via a printed QR code (demonstrated with a sample patient QR asset in the repository) was validated as a practical bridge for a patient's first login before they are comfortable navigating the MRN-entry flow alone.

8.8 Limitations of Current Testing Coverage

No formal clinical usability study with statistically powered participant recruitment has yet been conducted; testing to date has been developer- and pilot-staff-driven rather than an IRB-approved human-factors evaluation. The pilot remains single-site. As established in Chapter 7, the predictive model itself has no offline evaluation results to report because it has not yet been trained on a sufficient dataset — this is listed here again deliberately, as a testing-coverage limitation, rather than only as an implementation-status note, to keep the two framings from being read as contradictory.

CHAPTER 9

Results and Discussion

9.1 Delivered System Capabilities

By the September 2026 milestone documented in this thesis, the OPD AI Unit delivers, in active pilot use: full bilingual patient registration and record management; four-source pre-visit PROM capture across six validated instruments covering every body area the department treats; AI-assisted one-take clinical dictation with a deterministic multilingual safety layer; physician evaluation, diagnostic ordering, treatment recording, and a dedicated surgery-evaluation workflow; document upload and patient-facing retrieval; AI-generated plain-language patient visit summaries; follow-up scheduling and dual staff/patient notification inboxes; real two-way clinic messaging; a de-identified research export; and a fully independent mobile companion application giving patients OTP-secured access to their own visit history, PROM trend, documents, and clinic chat from a phone.

9.2 Mapping Objectives to Outcomes

Table 9.1 — Secondary objectives (Section 1.5) against delivered outcomes

Objective	Status
Encounter-centered longitudinal data model	ACHIEVED
Bilingual clinical web platform covering the full outpatient pathway	ACHIEVED
Mobile companion application	ACHIEVED
Reproducible de-identified research export	ACHIEVED
Offline training pipeline with leakage controls and sample-size guard	ACHIEVED
Baseline-vs-CatBoost comparison with reported metrics	PENDING — INSUFFICIENT CLINICALLY REVIEWED DATA
Audit trail for every AI suggestion and clinician response	ACHIEVED
AI-assisted dictation and AI-generated patient summaries	ACHIEVED

Six of eight secondary objectives are fully achieved and in live pilot use; the remaining objective — reporting comparative model metrics — is honestly reported as pending, for the reasons detailed in Chapter 7, with a concrete path to closing that gap specified in Section 7.9 rather than left open-ended.

9.3 AI Training Outcomes, Revisited

The primary AI-related result this thesis reports is not a predictive-performance number but a validated, leakage-safe, auditable, and honestly-gated pipeline — end-to-end engineering readiness, deliberately not yet exercised against a clinically sufficient dataset. This is a materially different, and in the authors' view methodologically preferable, outcome to a model trained prematurely on thirty pooled, unreviewed rows and reported with inflated confidence. The reusable contribution at this stage is architectural: an encounter boundary, a feature-scoping

discipline verified identical between training and inference, a hard minimum-sample refusal guard, and a safe-unavailable API contract that other small clinical datasets facing the same "too little data, too much at stake" problem could adopt directly.

9.4 Discussion

The encounter-centered redesign is best understood as the enabling precondition for everything in Chapter 7, not a side effect of it — without it, none of the leakage-prevention claims in Section 7.3 could be made with confidence. Independently of the predictive layer's current status, the dictation and patient-summary AI features already deliver measurable value today: they reduce the manual transcription burden on physicians during a time-constrained consultation, and they give patients a level of plain-language access to their own diagnosis and plan that the department's prior paper-based workflow never offered. The mobile app's existence and its reuse of backend logic with zero duplication is itself evidence that the underlying API design is sound rather than accidentally web-specific — a second, independently built client succeeding on the first attempt is a stronger validation of an API's generality than any amount of internal documentation would be.

9.5 Threats to Validity and Limitations

- **Single-site pilot.** All results and workflow observations come from one hospital department; generalization to other institutions' workflows is untested.
- **Short data-collection window.** The encounter model has been live only since roughly August 2026 at the time of writing, well short of the volume Section 7.9 argues is needed for a clinically defensible model.
- **Language coverage.** The system supports Arabic and English (with limited French keyword coverage in dictation); other languages spoken by patients at the site are not yet supported.
- **Observational confounding.** Any future outcome-effectiveness analysis built on this data will face confounding by indication, as discussed in Section 7.10, and must be designed accordingly rather than treated as causal by default.
- **Single-developer engineering risk.** The system was built and is maintained by a small team, which concentrates both institutional knowledge and delivery risk relative to a larger engineering organization.
- **Third-party AI dependency.** Dictation transcription, structured extraction, and patient-summary generation all depend on the OpenAI API, introducing cost, availability, and data-residency considerations — audio and transcript content leave the local machine for these calls, a tradeoff documented directly in the project's own `DICTATION_SETUP.md` and one that should be revisited if data-governance requirements tighten.

CHAPTER 10

Conclusion and Future Work

10.1 Summary of Contributions

This thesis delivered an encounter-centered orthopedic clinical data model implemented as a production schema; a complete, bilingual, role-based clinical web platform in active pilot use covering registration through follow-up; a cross-platform mobile companion application that reuses the web platform's backend and computation logic with no duplicated business logic; a safety-first AI dictation pipeline pairing deterministic multilingual keyword matching with LLM-based structured extraction; a reproducible, de-identified longitudinal research-dataset export; an offline, leakage-aware, minimum-sample-gated model-training pipeline for treatment-choice prediction; an auditable clinician-feedback loop wired into the physician's existing workflow; and a literature-grounded, clinically actionable deployment roadmap for the predictive layer, produced specifically because that layer is not yet trained on sufficient data.

10.2 Answering the Research Questions

- **RQ1** (encounter-centered dataset feasibility) — **Answered, positively.** The encounter model is live in production and correctly bounds every downstream clinical record without disrupting the existing pilot workflow.
- **RQ2** (does PROM data improve treatment-choice prediction) — **Not yet answerable.** Requires the sample-size threshold specified in Section 7.9 to be reached first; the feature pipeline needed to test it is built and verified.
- **RQ3** (best model architecture) — **Not yet answerable** for the same reason; the comparison protocol (Section 3.6) is specified and ready to execute once data sufficiency is reached.
- **RQ4** (explanations, confidence, and abstention surfaced usefully) — **Answered, positively, at the architecture level.** The Physician Evaluation UI panel, confidence display, warning surfacing, and four-way feedback control are built and live; their effect on real clinician behavior awaits the silent-mode validation recommended in Section 7.9.
- **RQ5** (governance and validation conditions for safe promotion to live use) — **Answered.** Section 7.9 specifies concrete sample-size, calibration, subgroup-audit, silent-mode, ethics, and governance conditions.

10.3 Practical Impact for the Host Clinical Site

Independent of the predictive model's current status, the host department gained a unified digital pathway from registration through follow-up, standardized and validated PROM scoring across six instruments where informal or inconsistent scoring previously existed, a documentation workflow that measurably reduces manual transcription during consultation, and — for the first time — a direct channel giving patients plain-language visibility into their own diagnosis, plan, and progress between visits via the mobile companion app.

10.4 Future Work

NEAR-TERM

- Continue encounter data collection toward the per-body-area sample-size targets specified in Section 7.9, with clinician-reviewed treatment-taxonomy labels.

- Train and evaluate the majority-baseline, logistic-regression, random-forest, and CatBoost comparison specified in Section 3.6 once that threshold is reached, reporting macro-F1, top-3 recall, calibration, and subgroup performance in full.
- Run the silent-mode (shadow) validation protocol before enabling any live suggestion in the physician-facing UI.
- Produce a versioned model card per promoted artifact, per the governance process recommended in Section 7.9.

MEDIUM-TERM

- Move from a single pooled treatment-choice model to per-body-area models, evaluated independently.
- Implement real device push notifications via an `expo-dev-client` build, using the server-side trigger point already present in `backend/notifications.py`.
- Add backend-side storage for per-patient notification and language preferences, currently local-only UI state in the mobile app.
- Begin a properly confounding-aware treatment-outcome effectiveness study once `TreatmentOutcome` completeness improves under the standardized follow-up windows recommended in Section 7.9 — explicitly distinct from, and a harder claim than, the treatment-choice prediction task this thesis addresses.

LONG-TERM

- Extend validation beyond the single pilot site once results there are mature, to test generalization of both the workflow design and any trained model.
- Pursue a prospective, IRB-approved evaluation pathway for any live-suggestion deployment, rather than relying solely on retrospective validation.
- Report any eventual predictive-model study using a current, prediction-model-specific reporting standard such as TRIPOD+AI, to keep the eventual clinical claims held to the same rigor this thesis has applied to the claims it declines to make today [21].

10.5 Closing Statement

This thesis set out to connect patient-reported data, clinical history, physician documentation, and treatment decisions into a single, explainable, encounter-centered clinical system — one in which the physician remains the final decision-maker at every step, and in which the machine-learning layer improves access to relevant information and communicates uncertainty rather than displacing judgment. The web platform and its independent mobile companion demonstrate that this architecture works as a piece of real clinical infrastructure, already in daily pilot use. The predictive treatment-recommendation layer built on top of that architecture is, as of this writing, engineering-complete and deliberately silent — a considered result of the same safety-first discipline that shaped every other decision in this system, and the concrete starting point for the work that follows.

References

[1] Sutton, R. T., Pincock, D., Baumgart, D. C., Sadowski, D. C., Fedorak, R. N., & Kroeker, K. I. (2020). An overview of clinical decision support

systems: benefits, risks, and strategies for success. *npj Digital Medicine*, 3(1), 17.

[2] Topol, E. J. (2019). High-performance medicine: the convergence of human and artificial intelligence. *Nature Medicine*, 25(1), 44–56.

[3] Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. *New England Journal of Medicine*, 380(14), 1347–1358.

[4] Roos, E. M., & Lohmander, L. S. (2003). The Knee injury and Osteoarthritis Outcome Score (KOOS): from joint injury to osteoarthritis. *Health and Quality of Life Outcomes*, 1, 64.

[5] Nilsdotter, A. K., Lohmander, L. S., Klässbo, M., & Roos, E. M. (2003). Hip disability and osteoarthritis outcome score (HOOS): validity and responsiveness in total hip replacement. *BMC Musculoskeletal Disorders*, 4, 10.

[6] Beaton, D. E., Wright, J. G., & Katz, J. N. (2005). Development of the QuickDASH: comparison of three item-reduction approaches. *Journal of Bone and Joint Surgery (American)*, 87(5), 1038–1046.

[7] Fairbank, J. C., & Pynsent, P. B. (2000). The Oswestry Disability Index. *Spine*, 25(22), 2940–2952.

[8] Vernon, H., & Mior, S. (1991). The Neck Disability Index: a study of reliability and validity. *Journal of Manipulative and Physiological Therapeutics*, 14(7), 409–415.

[9] Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data? *Advances in Neural Information Processing Systems*, 35.

[10] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.

[11] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

[12] Van Calster, B., McLernon, D. J., van Smeden, M., Wynants, L., & Steyerberg, E. W. (2019). Calibration: the Achilles heel of predictive analytics. *BMC Medicine*, 17, 230.

[13] Riley, R. D., Ensor, J., Snell, K. I. E., Harrell, F. E., Martin, G. P., Reitsma, J. B., Moons, K. G. M., Collins, G., & van Smeden, M. (2019). Minimum sample size for developing a multivariable prediction model. *BMJ*, 368, m441.

[14] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. *Proceedings of the 40th International Conference on Machine Learning*.

[15] Marcolino, M. S., Oliveira, J. A. Q., D'Agostino, M., Ribeiro, A. L., Alkmim, M. B. M., & Novillo-Ortiz, D. (2018). The impact of mHealth interventions: systematic review of systematic reviews. *JMIR mHealth and uHealth*, 6(1), e23.

[16] Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.

[17] Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77.

[18] Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

[19] Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: a framework for evaluation in design science research. *European Journal of Information Systems*, 25(1), 77–89.

[20] World Health Organization. (2021). *Ethics and governance of artificial intelligence for health: WHO guidance*. World Health Organization.

[21] Collins, G. S., Moons, K. G. M., Dhiman, P., et al. (2024). TRIPOD+AI statement: updated guidance for reporting clinical prediction models that use regression or machine learning methods. *BMJ*, 385, e078378.

Appendices

Appendix A — Representative API Endpoint Inventory

Grouped by router as registered in `backend/main.py`; representative, non-exhaustive within each module.

Router	Representative endpoints	Access
auth	staff login / token issuance	public
patients	patient CRUD, search	staff
assessments	pre-visit PROM records	staff
evaluations	physician evaluation, “send to patient”	staff
surgery_evaluations	operative documentation	staff, non-nurse
diagnostics / treatments	diagnostic orders, treatment records	staff
documents	upload / retrieve visit documents	staff
encounters	`POST /api/encounters`	staff
research	`GET /api/research/encounters.csv`	staff
ai_assistant	`GET /api/ai/status`, `POST /api/ai/patients/{id}/recommendation`, `POST /api/model-suggestions/{id}/feedback`	staff
outcomes	`POST /api/patients/{id}/treatment-outcomes`	staff
prom_assignments / prom_trend	PROM scheduling, trend computation	staff
prom_public	tokenized patient self-fill form	public token
followups / staff_notifications /	follow-up calls, staff inbox, clinic messaging	staff

Router	Representative endpoints	Access
chat		
patient_auth	`POST /request-otp`, `POST /verify-otp`	public
patient_self	`GET /me`, `/visit-reports`, `/notifications`, `/appointment`, `/active-care`, `/prom-status`, `/prom-trend`, `/messages`	patient

Appendix B — Data Model Entity Groups

Group	Entities
Core clinical	`Patient`, `Encounter`, `Assessment`, `Evaluation`, `SurgeryEvaluation`, `Diagnostic`, `Treatment`, `TreatmentOutcome`, `Document`, `InjuryEvent`
Configuration / admin	`User`, `AssessmentConfig`, `StatusConfig`, `FollowUpSettings`, `DiagnosticTestOption`, `TreatmentOption`
Scheduling / outreach	`FollowUpCall`, `PROMAssignment`
AI & research audit	`ModelSuggestion`, `ModelFeedback`
Patient-facing / communication	`PatientOTP`, `PatientNotification`, `StaffNotification`, `ChatMessage`

Appendix C — PROM Instrument Scoring Reference

See Table 5.2 for the full instrument list. Source scoring manuals used for implementation verification (Section 8.2) are retained in the project's `scores/` directory, including the KOOS and HOOS official documentation and neck/NDI scoring notes.